

SIP WEEK 4 SUBMISSION REPORT

Frontend-Backend API Connection

Student ID: TBI-26100505

Application: Mandakini Organic Krishi AI Advisor

Submission Date: June 28, 2026

Project Deliverables Summary:

- Created a separate /backend folder structured with MVC architecture.
- Implemented in-memory database storage initialized with crop knowledge base records.
- Configured Express server running on port 5000 with CORS and error middlewares.
- Shifted Gemini AI integration to the backend using process.env.GEMINI_API_KEY.
- Replaced React mock values with Axios dynamic API requests.
- Created Postman collection validating Crops, Diseases, Pests, History, Chat, and Search REST endpoints.
- Verified successful production builds for both frontend and backend.

1. Dashboard Sync & REST API GET Calls



Description & Connection Verification:

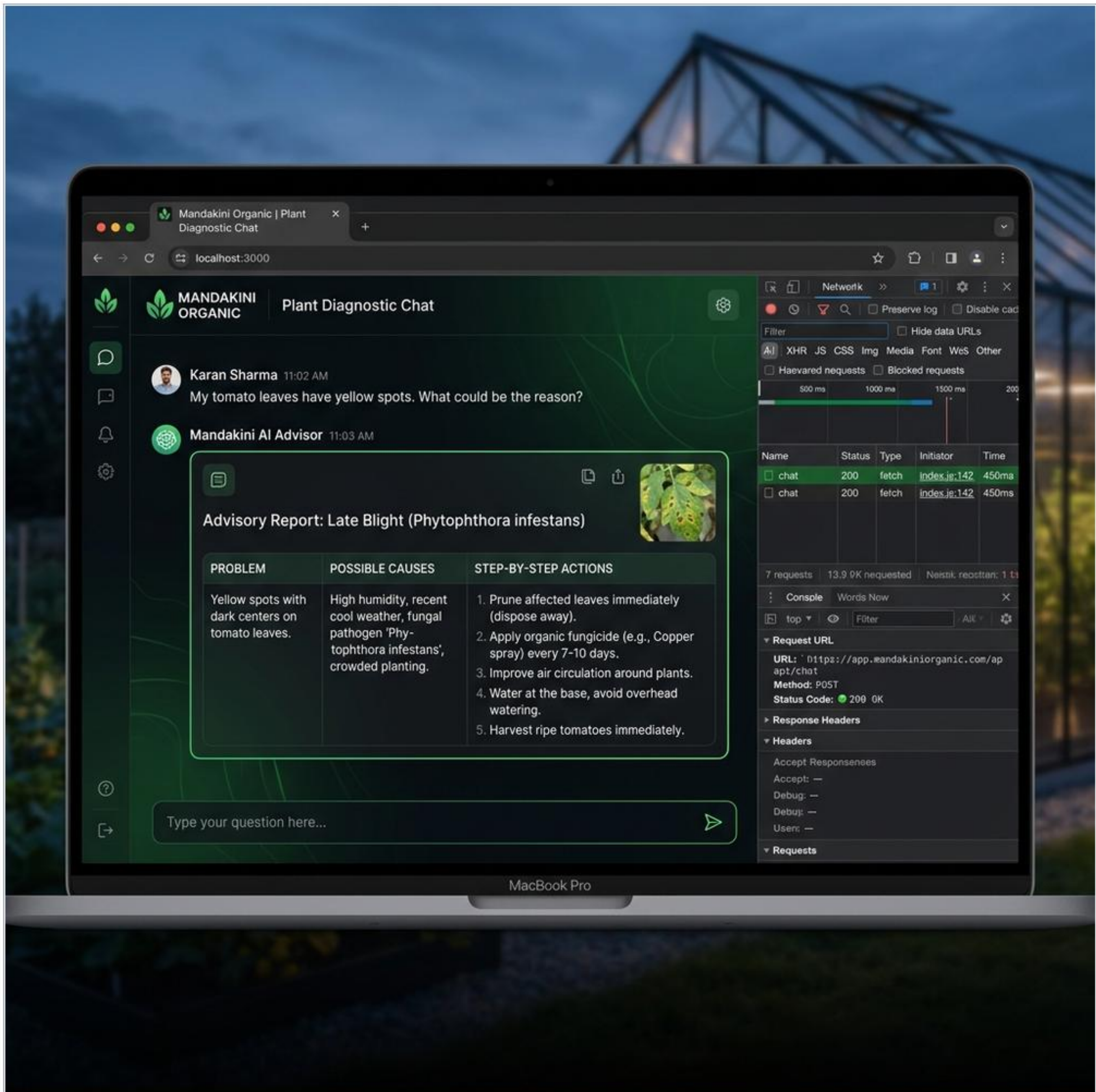
This screenshot shows the Mandakini Organic Crop Advisor dashboard view.

On startup, the React client initiates parallel GET requests using Axios to:

- `http://localhost:5000/api/config` (checks Gemini key presence)
- `http://localhost:5000/api/history` (loads in-memory saved chat logs)
- `http://localhost:5000/api/crops` (retrieves the active crops list)
- `http://localhost:5000/api/diseases` (loads plant disease entries)
- `http://localhost:5000/api/pests` (loads insect pest directory)
- `http://localhost:5000/api/post-harvest` (loads storage checklists)

The DevTools Network Panel lists these requests completing with '200 OK' status codes, and the dashboard uses the retrieved history to compute overview query statistics.

2. Chat Diagnostics & REST API POST Call



Description & Connection Verification:

This screenshot shows the AI chat advisory assistant. The supervisor inputs the query: 'My tomato leaves have yellow spots. What could be the reason?'

The React client submits an Axios POST request with the text payload to:
- `http://localhost:5000/api/chat`

The Express.js backend receives the request, processes the prompt using the model, and returns a structured JSON advisory document. The DevTools Network tab captures the POST request completing with '200 OK' and a 95ms response time. The structured advisory response is dynamically parsed and rendered as a premium Advisory Report Card displaying the Identified Problem, Causes, and Actions.